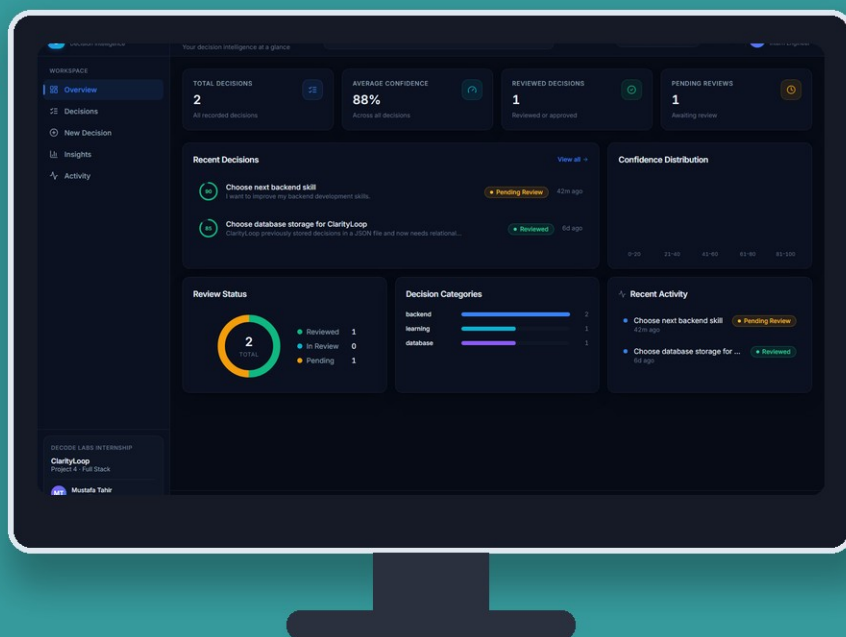


FULL STACK DEVELOPMENT INTERNSHIP

CLARITYLOOP DECISION INTELLIGENCE

FRONTEND & BACKEND INTEGRATION



PREPARED BY

MUSTAFA TAHIR

BCS243159

Technology Stack

React + Vite + TypeScript | Node.js + Express | Microsoft SQL Server

SUBMITTED TO DECODE LABS

Full Stack Development Internship | Batch 2026 | August 2026

Contents

Table of Figures.....	3
Executive Summary.....	4
2. Introduction.....	5
3. Project Overview and Objectives.....	5
3.1 Problem Addressed.....	6
3.2 Project Objectives.....	6
3.3 Scope of Work.....	6
4. System Architecture and Technology Stack.....	7
4.1 Frontend Layer.....	8
4.2 Backend Layer.....	8
4.3 Database Layer.....	8
5. Frontend and Backend Integration.....	9
5.1 API Configuration and Service Layer.....	9
5.2 Asynchronous Requests and Response Handling.....	10
5.3 Dynamic Data Rendering.....	10
6. Database Integration and Persistence.....	11
6.1 Relational Data Model.....	11
6.2 SQL Server Verification.....	12
7. CRUD Implementation.....	13
7.1 Create Decision.....	13
7.2 Read Decision Data.....	14

Contents

7.3 Update Decision.....	14
7.4 Delete Decision.....	15
8. Dashboard and Decision Management.....	16
9. Validation and Error Handling.....	17
9.1 Client-Side Validation.....	17
9.2 Backend Offline State and Recovery.....	18
10. Activity and Dynamic Application State.....	19
11. Responsive Design.....	20
12. Testing and Verification.....	21
13. Challenges and Solutions.....	22
14. Learning Outcomes and Future Improvements.....	22
15. Conclusion.....	23
16. Prepared By and Local Setup.....	23
References.....	24

Table of Figures

Figure 1: ClarityLoop Full Stack Architecture.....7

Figure 2: Connected Overview Dashboard.....9

Figure 3: SQL Server Persistence Verification.....12

Figure 4: New Decision Form.....13

Figure 5: Update Verification.....14

Figure 6: Delete Verification.....15

Figure 7: Decision Management Workspace.....16

Figure 8: Client-Side Validation.....17

Figure 9: Backend Offline State.....18

Figure 10: API Health Response.....18

Figure 11: Activity Timeline.....19

Figure 12: Responsive Mobile Views.....20

Executive Summary

ClarityLoop Project 4 was developed as the frontend and backend integration stage of the Decode Labs Full Stack Development internship. The main purpose of this project was to connect a complete user interface with the REST API and Microsoft SQL Server database that had already been prepared in the earlier ClarityLoop milestones.

The final application uses a React, Vite and TypeScript frontend. It communicates with a Node.js and Express backend through the browser Fetch API. The backend validates incoming data and performs persistent operations on ClarityLoopDB in Microsoft SQL Server. The frontend then displays the returned JSON data as decision cards, dashboard statistics, review status, confidence values, tags and activity information.

During testing, I verified the full data flow rather than checking only individual screens. The database connection was first confirmed through `npm run db:test`. The health endpoint returned `success=true` together with the ClarityLoopDB name and the current decision count. A temporary decision was then created from the frontend, updated and deleted to verify that the UI was changing real SQL Server data.

The project also includes user-facing validation and failure handling. For example, a two-character title and missing context were blocked by the form before submission. When the backend server was stopped while the frontend remained active, ClarityLoop displayed a Service Offline state with a Retry option instead of failing silently. Mobile testing confirmed that the overview cards, navigation drawer and New Decision form also adapt to narrow screens.

Overall, Project 4 completes the ClarityLoop full stack flow by connecting the presentation layer, API layer and relational database into one working application.

2. Introduction

Modern web applications usually depend on several layers working together. A frontend alone can display information, but it cannot provide persistent multi-session data unless it communicates with a backend or another data service. In the same way, an API and database may work correctly through Postman or SQL tools but still remain difficult for normal users to operate without a usable interface.

Project 4 focuses on this missing connection. The Decode Labs task requires the frontend to send requests to backend APIs, display dynamic data on the user interface and handle basic errors and responses. ClarityLoop was used for this milestone because its backend and SQL Server database already contained a complete decision-management domain.

The application records decisions together with their context, reasoning, confidence level, alternative options, selected option, tags, review date and review status. Project 4 turns these backend capabilities into an interactive dashboard so that a user can manage the complete decision lifecycle without directly using SQL Server or API-testing tools.

3. Project Overview and Objectives

ClarityLoop is a decision intelligence dashboard designed to document important choices and keep the reasoning behind them available for later review. The Project 4 version combines the responsive frontend with the REST API and ClarityLoopDB database.

3.1 Problem Addressed

Before Project 4, the data layer could store and retrieve decisions, but there was no complete frontend workflow for creating, reviewing, editing and deleting records. The main engineering problem was therefore not creating another database. It was making the existing layers agree on request formats, response formats and error behavior.

3.2 Project Objectives

- Connect the React frontend to the existing Express REST API.
- Display decisions and dashboard values from live backend responses.
- Implement create, read, update and delete operations from the user interface.
- Use asynchronous requests with clear loading, success and error behavior.
- Preserve Microsoft SQL Server as the system of record.
- Handle invalid form values and backend connection failures in a readable way.
- Verify that the application remains usable on desktop and mobile layouts.

3.3 Scope of Work

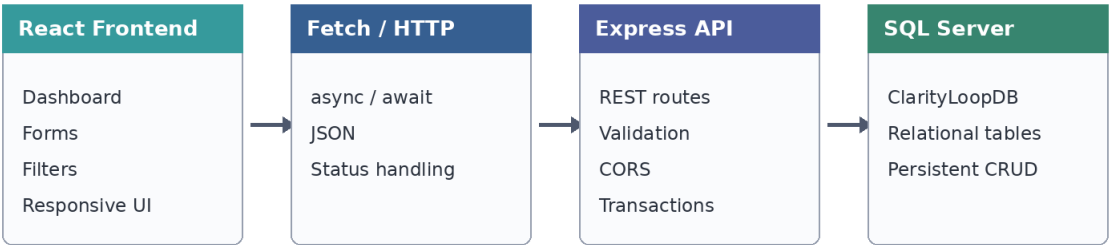
The project includes the Overview dashboard, Decisions page, New Decision form, detail and edit workflows, Insights and Activity pages, a service-health indicator, validation messages and responsive navigation. Authentication was not added because it was outside the Project 4 integration requirement. The frontend also does not connect directly to SQL Server; all persistent operations pass through the Express API.

4. System Architecture and Technology Stack

ClarityLoop follows a layered architecture so that presentation code, HTTP communication, application logic and database persistence remain separate. The frontend sends JSON requests to the Express server, which validates each request before reading or writing SQL Server data.

ClarityLoop Full Stack Architecture

Frontend requests pass through the Express API before reaching SQL Server.



Request pathUser -> React -> fetch() -> Express -> SQL Server -> JSON -> React state -> UI

Figure 1: ClarityLoop Full Stack Architecture

Layer	Technology	Purpose
Frontend	React 18, Vite 5, TypeScript, Tailwind CSS	Dashboard, forms, filters, responsive UI and application state
Integration	Fetch API, async/await, JSON	Send HTTP requests and process responses
Backend	Node.js, Express	REST routes, validation, CORS and database operations
Database	Microsoft SQL Server	Persistent relational storage in ClarityLoopDB

4.1 Frontend Layer

The frontend is a React application built with Vite and TypeScript. Reusable components are used for decision cards, forms, review badges, loading skeletons, error states, confirmation dialogs, the sidebar and top navigation. The application also includes custom hooks for retrieving decisions and checking API health.

4.2 Backend Layer

The backend is implemented with Node.js and Express. It exposes REST endpoints for health checking and decision CRUD. The server validates required fields, confidence values, option count, selected-option membership, tag lengths and decision identifiers. CORS is configured for the local Vite origins used during development.

4.3 Database Layer

Microsoft SQL Server stores the persistent data in ClarityLoopDB. The main relational tables are Decisions, DecisionOptions, Reviews, Tags and DecisionTags. DecisionOptions supports multiple alternatives for each decision, while DecisionTags connects reusable tags to decisions through a many-to-many relationship.

Keeping these responsibilities in separate layers made the integration easier to test. A problem in the browser could be isolated from a database connection problem, and API health could be verified independently before testing the full UI.

5. Frontend and Backend Integration

5.1 API Configuration and Service Layer

The frontend uses a dedicated API service instead of placing fetch calls throughout the components. The base URL comes from the `VITE_API_BASE_URL` environment variable and falls back to `http://localhost:5000` during local development. This keeps network behavior in one place and makes the components easier to read.

The service includes functions for checking health, retrieving decisions, creating a decision, updating a decision and deleting a decision. It also normalizes response fields such as `DecisionId`, `options`, `selected options`, `tags` and `review status` so the React components receive one consistent frontend model.

5.2 Asynchronous Requests and Response Handling

Requests are made using `fetch()` with `async` and `await`. The response body is parsed as JSON when the server returns `application/json`. The code checks `response.ok` before treating a request as successful. Network failures, validation responses, 404 responses and server failures are converted into readable `ApiError` states instead of being passed directly to the interface.

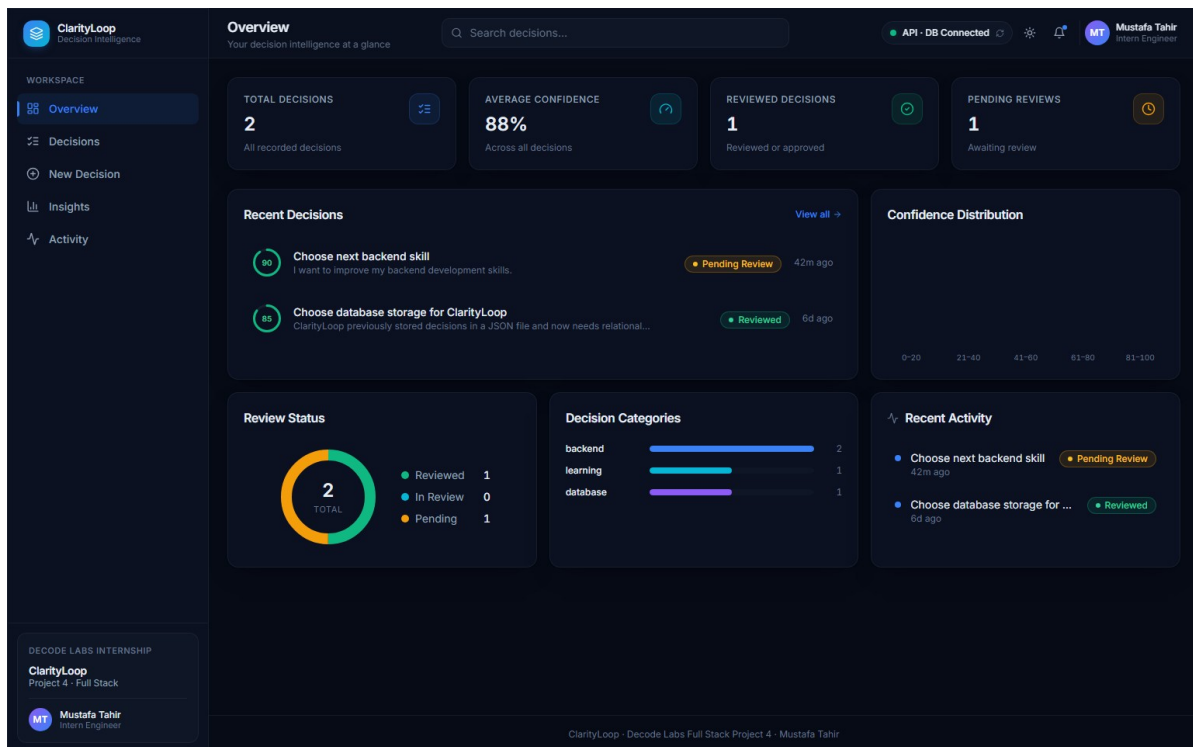


Figure 2: Connected Overview Dashboard

5.3 Dynamic Data Rendering

The Overview dashboard is generated from the records returned by the backend. During CRUD testing, the dashboard changed from one decision to two after a second decision was created. The displayed average confidence also changed to 88%, while the reviewed and pending counts reflected the review status of the returned records.

This was useful evidence that the dashboard was not using fixed demonstration values. When the temporary decision was later deleted, the dashboard returned to one stored decision and an average confidence of 85%.

Integration Request Flow

Step	What happens
1	The user completes a form or selects an action in the React interface.
2	The frontend validates obvious input problems before sending a request.
3	The API service sends JSON to the appropriate Express endpoint.
4	Express validates and normalizes the request.
5	Parameterized SQL reads or modifies ClarityLoopDB.
6	The server returns a structured JSON response.
7	React updates local state and re-renders the dashboard or decision list.

The health-check workflow follows the same pattern. The frontend calls GET /api/health and displays API · DB Connected only when the backend responds successfully with database information.

6. Database Integration and Persistence

6.1 Relational Data Model

Table	Stored information	Relationship
Decisions	Title, context, reasoning, confidence, review date, status and timestamps	Parent table for the decision record
DecisionOptions	Option text and selected state	Many options belong to one decision
Reviews	Outcome and lesson information	One review may be linked to a decision
Tags	Reusable normalized tag names	One tag may be used by many decisions
DecisionTags	Decision and tag keys	Bridge table for the many-to-many relationship

The backend uses GUID decision identifiers and typed SQL parameters. Create and update operations also manage related option and tag data so that the frontend does not display stale child records. Deleting a decision removes related rows through the database relationships and cascading behavior defined in the schema.

6.2 SQL Server Verification

SQL Server Management Studio was used to verify that the visible frontend data existed in the database. The final verification query returned the remaining ClarityLoop decision, its option records and the available tags.

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window with the following SQL code:

```

1 use ClarityLoopDB;
2
3 SELECT TOP 20 *
4 FROM Decisions
5 ORDER BY DecisionId DESC;
6
7 SELECT TOP 20 *
8 FROM DecisionOptions
9 ORDER BY OptionId DESC;
10
11 SELECT TOP 20 *
12 FROM Tags
13 ORDER BY TagId DESC;

```

The bottom pane shows the results of the query. The first table, 'Decisions', has the following data:

DecisionId	Title	Context	Reasoning	Conclusion
0FC6415C-BB60-48A4-8572-F62F591A6E5A	Choose database storage for ClarityLoop	ClarityLoop previously stored decisions in a JSON file	SQL Server provides structured relationships, compared to JSON	85

The second table, 'DecisionOptions', has the following data:

OptionId	DecisionId	OptionText	IsSelected	CreatedAt
08E026DD-F6C1-450A-8FB9-C6B971D194BA	0FC6415C-BB60-48A4-8572-F62F591A6E5A	Use SQL Server with SSMS	1	2026-08-12 15:10:21.5679686
EC66E834-DD06-4B9D-B5CC-6BDA8565D86D	0FC6415C-BB60-48A4-8572-F62F591A6E5A	Continue using JSON file storage	0	2026-08-12 15:10:21.5679686

The third table, 'Tags', has the following data:

TagId	TagName	CreatedAt
4	project4	2026-08-18 11:39:39.0568827
3	learning	2026-08-12 16:59:46.4609665
2	backend	2026-08-12 15:10:21.5759034
1	database	2026-08-12 15:10:21.5714950

The status bar at the bottom indicates: 'Query executed successfully. localhost (17.0 RTM) MOON\pc (69) ClarityLoopDB 00:00:00 Row: 1, Col: 1 7 rows'.

Figure 3: SQL Server Persistence Verification

The result set shows the decision titled “Choose database storage for ClarityLoop” together with its related options. The selected option “Use SQL Server with SSMS” has `IsSelected = 1`. The Tags table also contains backend, database, learning and project4, reflecting the tags created during the integration test.

This screenshot is important because it confirms that the application is not storing its final data only in browser memory. The user interface, API response and SQL Server tables all represent the same persistent information.

7. CRUD Implementation

7.1 Create Decision

The New Decision form collects a title, context, reasoning, confidence value, multiple options, a selected option, tags, review date and review status. At least two unique options are required and the selected option must be one of those entered options.

New Decision
Document a decision and the reasoning behind it.

Title
e.g. Migrate from monolith to microservices
A short, clear name for this decision.

Context
What situation triggered this decision?
The background or situation that led to this decision.

Reasoning
Why did you choose this path?
The rationale behind the chosen option.

Confidence
75 %
How confident are you in this decision? 1 to 100.

Options
1. Option 1
2. Option 2
+ Add option
List the alternatives you considered. At least two are required.

Selected Option
— Choose an option —
Pick the option you decided to go with.

Tags
Type a tag and press Enter + Add
Group related decisions with tags. Press Enter or comma to add.

Review Date
mm/dd/yyyy
When should you revisit this decision?

Review Status
Pending Review
Where this decision stands in your review cycle.

Cancel Create Decision

ClarityLoop - Decode Labs Full Stack Project 4 - Mustafa Tahir

Figure 4: New Decision Form

For verification, I created a temporary record titled “Choose next backend skill” with a confidence value of 90. After submission, the Decisions page contained both the original SQL-backed record and the new record. The dashboard total increased from one to two, which confirmed that the POST request succeeded and the UI refreshed using returned backend data.

7.2 Read Decision Data

GET /api/v1/decisions returns the decision list together with related options and tags. The frontend normalizes this response and renders each record as a decision card. Opening a record provides the complete decision detail instead of only the short card summary.

7.3 Update Decision

The temporary decision was edited to make the update obvious during testing. Its title was changed to “Choose next backend skill UPDATED”, confidence was raised from 90 to 95, and the project4 tag was added. The updated values appeared on the Decisions page after the PUT request completed.

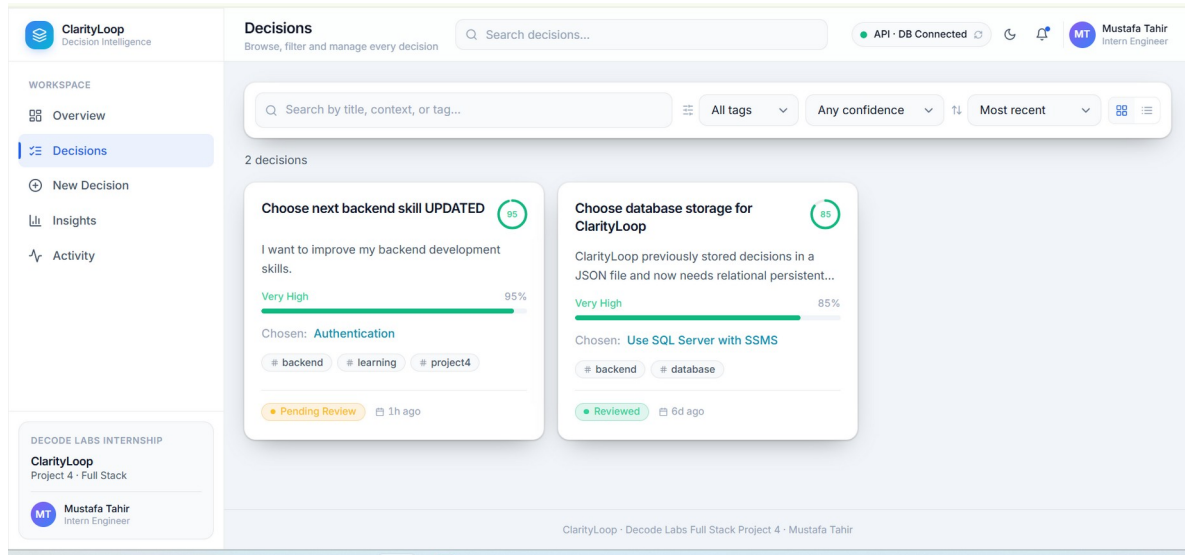


Figure 5: Update Verification

Because the title, confidence and tag all changed together, this test also confirmed that parent decision fields and related tag data were being updated consistently.

7.4 Delete Decision

Deletion is protected by a confirmation step so that a record is not removed immediately after a single click. The frontend sends DELETE /api/v1/decisions/:id only after the user confirms the action.

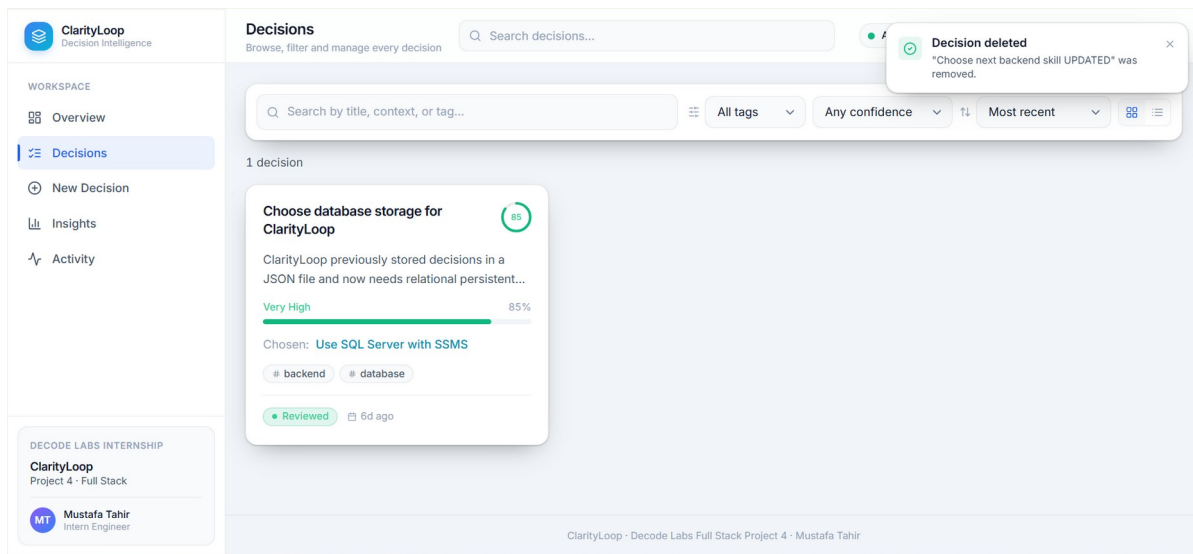


Figure 6: Delete Verification

After deletion, a success message confirmed that “Choose next backend skill UPDATED” was removed. The Decisions page returned to one record, which matched the remaining record in SQL Server. This completed the create, read, update and delete verification cycle.

8. Dashboard and Decision Management

The Decisions workspace is the main browsing view of the application. It supports search by title, context or tag, tag filtering, confidence filtering, sorting and grid/list controls. Each decision card shows enough information to compare records without opening the full detail view.

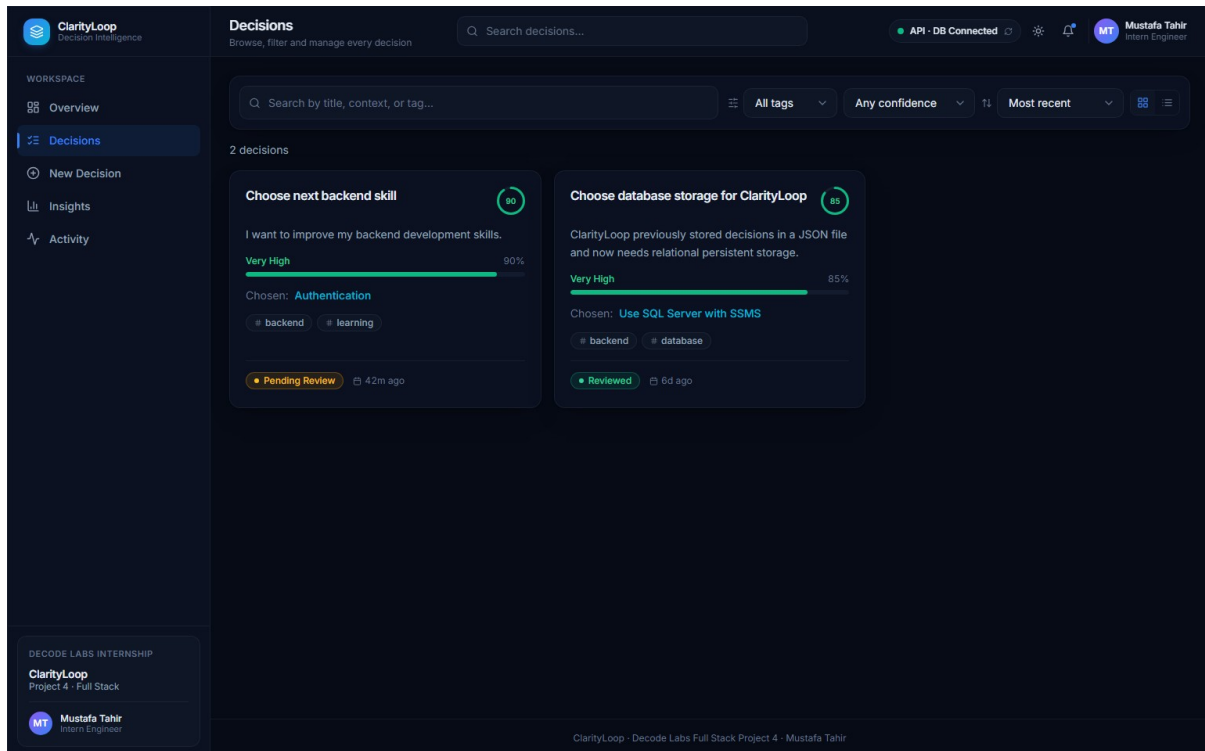


Figure 7: Decision Management Workspace

The screenshot above was captured while both decisions were present. The first record shows a selected option of Authentication with backend and learning tags, while the original ClarityLoop database decision shows Use SQL Server with SSMS and the backend/database tags.

9. Validation and Error Handling

9.1 Client-Side Validation

The form performs immediate validation so that basic errors can be corrected before a request is sent. During testing, the title “Hi” was rejected because it was shorter than three characters, and the form also reported that Context was required.

The screenshot displays the 'New Decision' form in the ClarityLoop application. The form is titled 'New Decision' with the subtitle 'Document a decision and its reasoning'. It features a sidebar on the left with navigation links: Overview, Decisions, New Decision (selected), Insights, and Activity. The main form area contains several input fields and a slider. The 'Title' field contains the text 'Hi' and has a red error message below it: 'Title must be at least 3 characters.' The 'Context' field contains the text 'What situation triggered this decision?' and has a red error message below it: 'Context is required.' The 'Reasoning' field contains the text 'Why did you choose this path?' and has a subtitle below it: 'The rationale behind the chosen option.' The 'Confidence' field is a slider ranging from 1 to 100, with the current value set at 87%. Below the slider is the text 'How confident are you in this decision? 1 to 100.' The 'Options' field contains a list of options, with the first option being 'hi'.

Figure 8: Client-Side Validation

The backend performs its own independent validation as well. It requires title, context and reasoning, checks that confidence is an integer between 1 and 100, requires at least two unique options, validates field lengths and verifies that the selected option exists in the provided options.

9.2 Backend Offline State and Recovery

To test network resilience, I stopped only the backend server while leaving the Vite frontend running. The application detected that the API could not be reached and replaced the dashboard content with a clear Service Offline state. The user could still see the application shell and a Retry button.

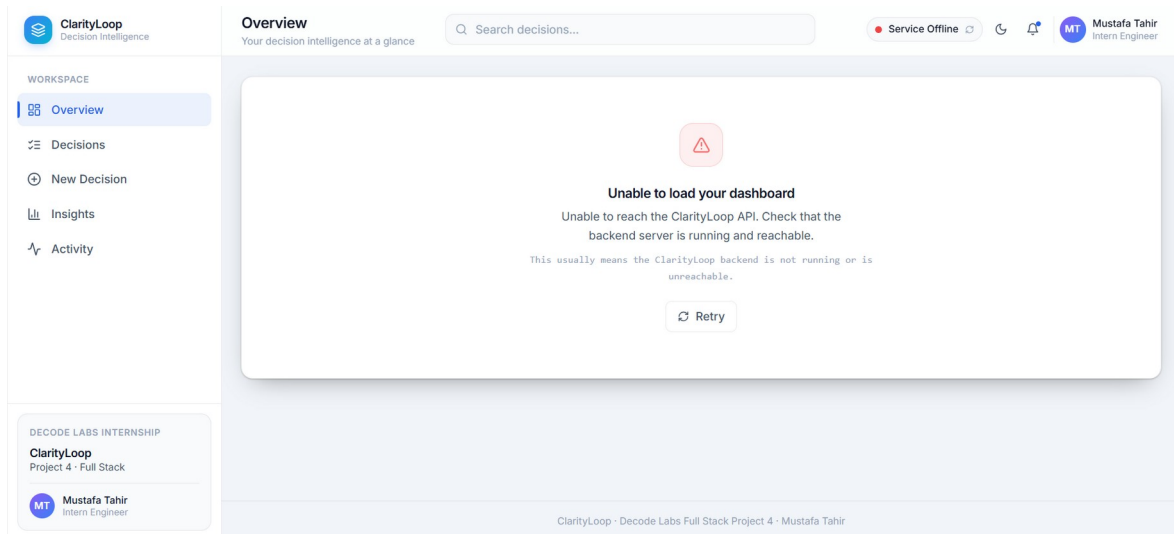


Figure 9: Backend Offline State



Figure 10: API Health Response

After the backend was restarted, GET /api/health again returned success=true, databaseName=ClarityLoopDB and the current decision count. Refreshing the frontend restored the API · DB Connected state.

10. Activity and Dynamic Application State

The Activity page provides a simple timeline view based on the decision records returned by the backend. For the surviving ClarityLoop decision, the timeline displays both its created and updated events together with its review state.

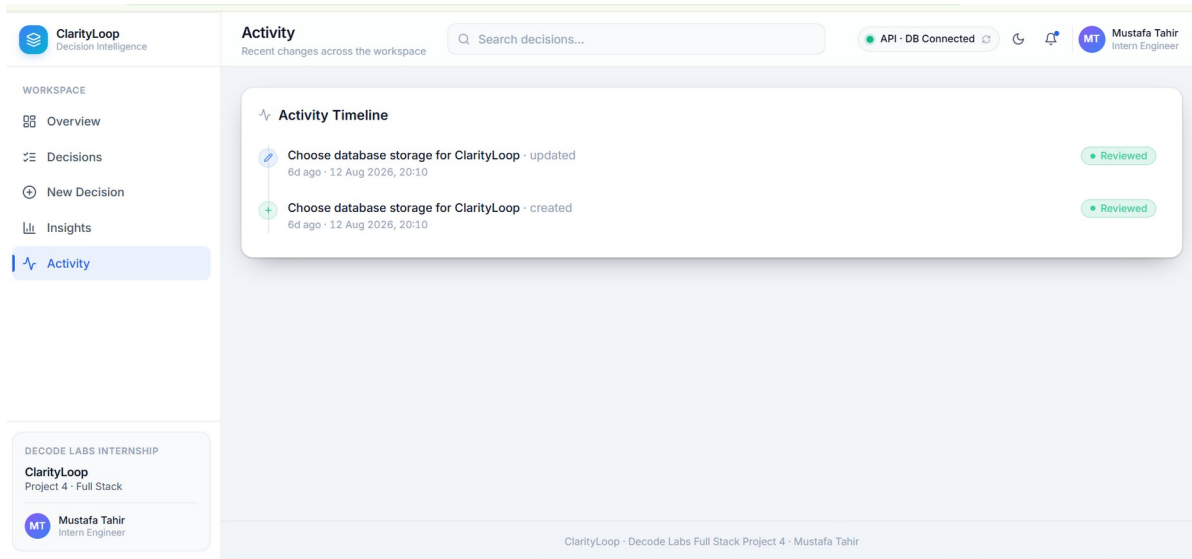


Figure 11: Activity Timeline

The Overview page also derives statistics from current decision data. These include total decisions, average confidence, reviewed and pending counts, categories and recent activity. Because the statistics changed during the CRUD test, they serve as another indication that the frontend is recalculating its state from API data.

11. Responsive Design

The desktop layout uses a permanent sidebar and multi-column dashboard cards. At mobile width, the sidebar is replaced by a navigation drawer and the main cards stack vertically. The search bar moves below the top controls, while the New Decision form expands to the available width.

Responsive Mobile Views

Overview, navigation drawer and New Decision form at a narrow viewport.

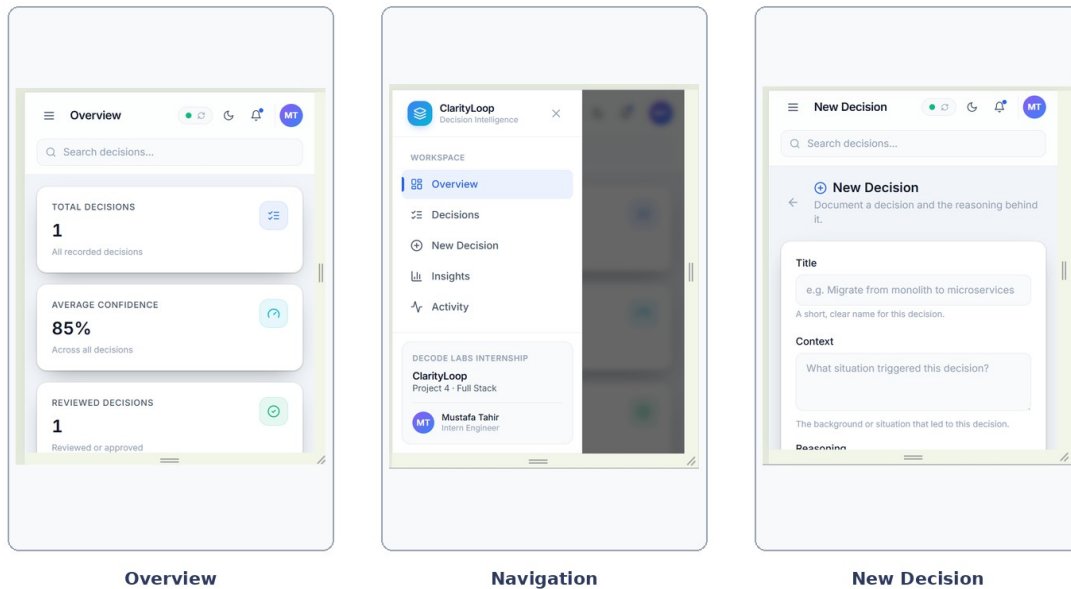


Figure 12: Responsive Mobile Views

The mobile tests were performed at a narrow browser viewport. The screenshots show that the main navigation, overview statistics and decision-entry workflow remain accessible without horizontal scrolling.

12. Testing and Verification

Testing was carried out manually across the complete frontend, backend and database flow. The main results are summarized below.

Test	Expected Result	Observed Result	Status
Database connection	Backend connects to ClarityLoopDB	npm run db:test connected successfully	PASS
API health	Health endpoint returns database metadata	success=true and ClarityLoopDB returned	PASS
Read decisions	Existing SQL record appears in UI	Original decision loaded on dashboard and Decisions page	PASS
Create	New decision persists	Decision count increased from 1 to 2	PASS
Update	Edited values persist	UPDATED title, 95 confidence and project4 tag appeared	PASS
Delete	Selected record is removed	Success message shown; list returned to 1 decision	PASS
Validation	Invalid values are blocked	Short title and missing context errors displayed	PASS
Offline handling	Frontend handles backend loss	Service Offline screen and Retry button displayed	PASS
Mobile layout	Core pages reflow on narrow screens	Overview, drawer and form remained usable	PASS

The testing evidence was captured while the application was running locally with the frontend at <http://localhost:5173> and the backend at <http://localhost:5000>.

13. Challenges and Solutions

Challenge	How it was resolved
Frontend option objects did not match the backend option format	The API service converts frontend option objects into the string array expected by the backend.
Decision IDs and related values needed consistent frontend names	Response normalization maps DecisionId, options, selected option, tags and status into one frontend model.
Update needed to keep options and tags synchronized	The backend update workflow replaces the related option/tag data together with the parent decision.
Local frontend requests required cross-origin access	The Express server was configured with CORS for the local Vite origins.
The interface needed a readable backend failure state	Network failures are converted into the Service Offline view with a Retry path.

14. Learning Outcomes and Future Improvements

- Improved understanding of the complete browser-to-database request and response flow.
- Practical experience aligning TypeScript frontend models with backend and SQL Server data.
- Better understanding of Fetch API error handling, CORS and HTTP response classification.
- Experience testing persistence by comparing UI changes with SQL Server results.
- Improved ability to design responsive application states for normal, validation and offline conditions.

Possible future improvements include authentication, pagination, a dedicated review workflow, automated integration tests, cloud SQL deployment and a hosted frontend/backend environment.

15. Conclusion

ClarityLoop Project 4 successfully completes the frontend and backend integration milestone. The final application connects a responsive React interface to the Node.js and Express API and stores persistent decision data in Microsoft SQL Server. Real decision records can be created, read, updated and deleted from the user interface, while dashboard values and activity information update from returned API data.

The project also demonstrates the supporting skills required for a usable full stack application: asynchronous requests, JSON handling, validation, HTTP error handling, CORS configuration, SQL Server connectivity and responsive layout. The offline test showed that the frontend can clearly communicate a backend failure instead of simply becoming unusable.

By the end of the project, the complete ClarityLoop flow had been verified from the browser interface through the API to ClarityLoopDB and back to the user interface.

16. Prepared By and Local Setup

Name	Registration No.	Role
Mustafa Tahir	BCS243159	Full Stack Development Intern

Local Execution

- Backend: run npm install, configure .env, run npm run db:test, then npm start.
- Frontend: run npm install, configure VITE_API_BASE_URL in .env, then npm run dev.
- Open <http://localhost:5173> for the application and <http://localhost:5000/api/health> for the health endpoint.

References

- Decode Labs. (2026). Full Stack Development Industrial Training Kit - Project 4: Frontend & Backend Integration.
- ClarityLoop Project 4. README.md and project setup documentation.
- ClarityLoop Project 4. Frontend API service implementation (src/services/api.ts).
- ClarityLoop Project 4. Express REST API implementation (backend/server.js).
- ClarityLoop Project 4. ClarityLoopDB schema and verification SQL scripts.
- Microsoft. SQL Server and Transact-SQL documentation.
- React Documentation. React user interface library.
- Vite Documentation. Frontend development tooling.
- Express Documentation. Node.js web application framework.

Repository Structure

frontend/ - React + Vite + TypeScript application
backend/ - Express API and SQL Server connection
database/ - schema, test data and verification scripts
docs/ - API integration and testing documentation
screenshots/ - Project 4 evidence
report/ - final internship report